



In-House Ring-All Call Routing (“Smart Queue”)

Delivered by: ASRP · **Role:** ASRP owned the call-routing subsystem end to end (design, implementation, roadmap) · **Engagement type:** Independent contractor · **Domain:** Cloud telephony for freight logistics

Реализовано: ASRP · **Роль:** ASRP полностью владела подсистемой маршрутизации вызовов (проектирование, реализация, roadmap) · **Тип сотрудничества:** независимый подрядчик · **Домен:** облачная телефония для грузовой логистики

1. Executive summary

ASRP designed and built an in-house **ring-all call-distribution system** — internally “Smart Queue” — for a cloud telephony platform serving freight logistics. When a call arrives on a routing-enabled number, the platform **rings every logged-in agent of the organization simultaneously**. The **first agent whose leg is answered claims the call**; all other legs are dropped, the caller and the winning agent are bridged, and if nobody answers a configurable fallback route fires.

The distinctive engineering property is **deterministic first-to-answer under a race**: two agents can answer within milliseconds of each other and exactly one wins, with no double-answer and no dropped caller. That reliability is the product of three mechanisms working together, not a single trick: every claim attempt is **serialized inside a Redis distributed lock (Redlock)**, so the first answer to acquire the lock wins and every later one is hung up cleanly; a **two-level feature gate** (organization and per-number) is **re-checked at three independent points** — call entry, claim, and fallback — so a mid-call configuration change can never strand a call on an inconsistent path; and the fallback route itself is made **idempotent** via a Redis deduplication key, so a retried or duplicate webhook can’t route the same caller twice. §5 and §11 walk through each mechanism in full.

Smart Queue is a **pilot** — a candidate to replace the platform’s incumbent external-PBX hunt-group. The core routing, claim, gating, presence, and fallback are built and exercised by an automated test suite; the primary hardening focus before a production cutover is answer-detection edge cases on outbound carrier routes (see §13–14). Notably, the current architecture is the *second* one ASRP built for this feature: an initial proof-of-concept was merged, then **fully reverted**, then rebuilt on a different design — the story in §8.

2. Business context & the problem

The platform routes inbound carrier calls to human agents, and it already had an in-house queue — a **first-generation, pull-model** one. A queued call was parked and surfaced to the whole organization in the platform UI; an available agent **clicked to claim** it, and the platform then dialed that agent and bridged the caller. It already leaned on a Redis lock so that if two agents clicked at nearly the same moment, exactly one won. That model worked for a team watching the screen, but it is fundamentally **reactive**: no phone rings on its own, so a call waits until a human notices it and clicks.

The business wanted the opposite ergonomics — **every available agent's phone ringing at once** the instant a call arrives, plus live presence and supervisor visibility over who is on a call and how each call was routed. That capability was first delivered by leaning on an **external cloud PBX** hunt-group. It gave the ring-and-route behavior, but it put the parts that matter most — queue behavior, agent presence, and which agent “got” the call — **outside** the platform, where they were hard to observe, extend, or reconcile with the platform's own real-time agent and supervisor UIs. Presence in the external PBX and presence in the platform UI became two sources of truth that could disagree.

The objective was to bring that ring-all behavior **back inside the platform** — a platform-native queue that:

- rings all available agents at once (minimal wait, no serial hunt down a list),
- lets the fastest agent win **deterministically**, with no double-answers,
- is driven by the platform's **own** database and agent-presence state (one source of truth),
- pushes real-time status to the agent and supervisor UIs, and
- falls back safely when no agent is available.

3. Why build it in-house (architecture rationale)

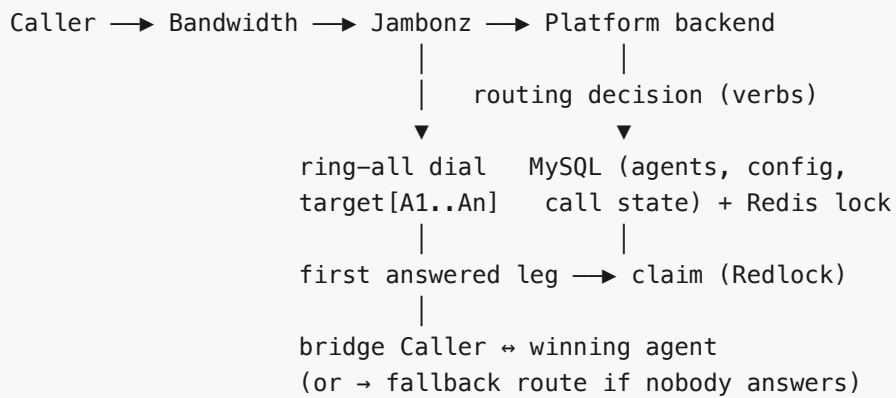
The obvious alternative was to keep leaning on the external PBX/ACD, or adopt another off-the-shelf contact-center product. ASRP's recommendation was to build the queue **inside** the platform, for concrete reasons rather than preference:

- **One source of truth for presence.** Agent login/logout already lived in the platform. An external hunt-group maintains its *own* presence, so the platform UI and the PBX drift out of sync. An in-house queue reads the same login state the UI shows.
- **Observability and control.** Every routing decision — who was eligible, who was rung, who won the race, why a fallback fired — is a structured event in the platform's own logs and database, not an opaque outcome from a third party.
- **The claim semantics are specific.** “Ring everyone, first *answer* wins, guarantee exactly one winner, drop the rest, fall back idempotently” is a precise contract. Owning the dial and the lock lets that contract be enforced and tested directly; delegating it to a PBX means inheriting whatever that PBX happens to do.
- **Extensibility.** Ring timeout, max agents, per-number enablement, custom messages, and future analytics are all reachable because the mechanism is code the team owns.

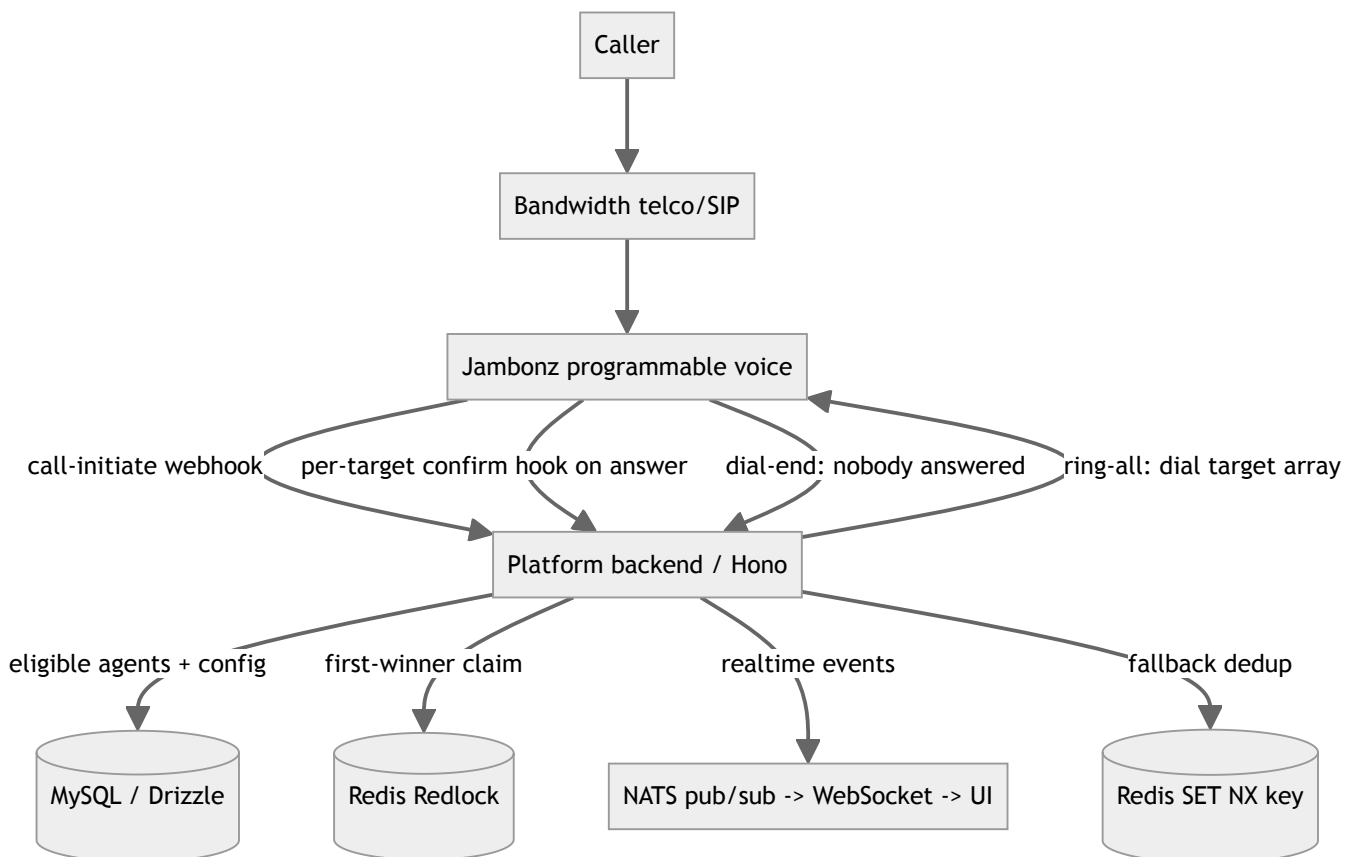
The tradeoff — accepted deliberately — is that the platform now owns hard telephony problems (simultaneous-answer races, answer supervision, fallback idempotency) that a PBX would have hidden. §8 and §14 are honest about where that bites.

4. System architecture

The call spine is `Caller → Bandwidth (telco/SIP) [^bandwidth] → Jambonz (programmable voice) → platform backend`. The backend makes the routing decision per call and returns Jambonz verbs; Jambonz executes the ring-all dial and calls back into the backend at each decision point (answer, dial-end).



Component roles



Component	Responsibility
Jambonz	Primary voice engine. Executes ring-all as one <code>dial</code> verb with a <code>target[]</code> array; invokes a per-target confirmation hook when a leg answers; bridges the winning legs; drives fallback via the dial-end action hook.
Bandwidth	Telco / SIP carrier. Auxiliary hold-audio verbs (BXML) are also supported.
Platform backend (TypeScript, Hono web framework)	Routing decisions, eligibility, claim, fallback, feature gating, event broadcast.
MySQL (Drizzle ORM)	Agent presence, per-user/per-number routing config, live call state.
Redis	Distributed lock (Redlock) for the claim race; a separate key for fallback deduplication.
NATS + WebSockets	Real-time queue / claim / agent-status events to the agent and supervisor UIs.

5. How the routing works

Ring-all is a **single Jambonz `dial` verb with a `target[]` array** and a per-target `confirmHook`.

Acceptance is **automatic on answer** via that hook, with no keypad step for the agent (see §8 for why). When an agent’s leg answers, the backend acquires a **Redis Redlock** keyed to the call and, inside the lock, records the first answerer as the winner and bridges the caller to them; any later answer finds the call already claimed and its leg is cleanly hung up. That lock is what guarantees exactly one winner even when two agents answer within milliseconds of each other. If nobody answers within the ring timeout, a dial-end hook fires an **idempotent** fallback route (§11).

The full step-by-step flow — the fixed routing order, the gating checkpoints, eligibility selection, the per-target hook payload, the exact claim / bridge / hangup responses, and the call-sequence diagram — is maintained in ASRP’s engineering reference. Deeper technical detail is available under NDA.

6. A sample call, in plain language

A carrier dials a broker’s number that has Smart Queue enabled. Two agents — call them **Priya** and **Sam** — are both logged in and eligible.

1. The call lands. The backend checks the two feature flags, finds Priya and Sam eligible, marks the call “in smart queue”, and tells Jambonz to ring **both** of their phones at once. Their UIs light up with the incoming call.
2. Both phones ring. Priya reaches for hers about **half a second** before Sam. She answers.
3. Priya’s answered leg fires her confirm hook. The backend takes the lock on this call, sees it is unclaimed, and writes “Call claimed by Priya”. It returns an empty response, and Jambonz bridges the caller straight to Priya. Her UI flips to “on call”.

4. A beat later Sam answers too. **His** confirm hook fires — but the backend takes the lock, sees the call is *already* claimed by Priya, and returns a hangup. Sam's leg drops cleanly; his UI clears. He never hears dead air or talks over Priya.
5. The caller and Priya are now talking. From the caller's side it was one ring and a pickup — they never knew two phones rang.

If neither Priya nor Sam had answered within the ring timeout, the dial would end without a bridge, the fallback path would fire exactly once (guarded against duplicate webhooks), and the caller would be routed to the configured fallback destination — or hear a graceful “unable to connect” message if none is set.

7. What ASRP built

- A **ring-all dial mechanism** as Jambonz programmable-voice verbs: a single `dial` verb targeting an array of agent destinations, each with its own per-target confirm hook.
- A **first-to-answer claim** protected by a Redis Redlock, so exactly one agent leg wins and the rest are hung up.
- A **two-level feature gate** (organization flag + per-number flag) checked consistently at call entry, at claim time, and on the fallback path.
- **Agent presence**: database-backed login/logout with a self-service toggle for agents and a management view for supervisors, plus a login-history archive.
- **Real-time events** pushed to the UI over NATS pub/sub and WebSockets (queue initiated, call claimed, agent status).
- **Safe fallback**: an idempotent path that routes the caller to a configured fallback destination, or plays a graceful message if none is configured.
- **tRPC procedures** and UI toggles for agent login and supervisor agent management.

8. Engineering story: the queue we shipped, killed, and rebuilt

The most instructive part of this project is that the architecture in §4–5 is **not** the first one that was built and merged.

Problem. Design a queue that rings agents and lets one of them take the call, deterministically.

First attempt (POC, merged). The initial proof-of-concept solved it with a **conference bridge plus an explicit “Press 1 to accept”**: rung agents joined a conference, and an agent pressed a key to confirm they were taking the call. It came with its own confirmation and outbound routes, a dedicated events table, a database migration, and a large (~2,400-line) design document. It was completed and **merged**.

The pivot. That entire merge was then **fully reverted** — the service, both confirmation routes, the outbound route, the migration, the events table, and the design doc were all removed in a single revert, and the codebase returned to having no Smart Queue at all. The reason was a delivery decision, not a technical dead-end: the same ring-all-and-route need was **already being met on the external-PBX path** built in parallel (§2), so rather than keep hardening a brand-new in-house voice mechanism, the team shipped on the PBX and **shelved** the in-house queue. It was set aside, not abandoned.

Rebuild (current design). The in-house approach was later **revived** and rebuilt from scratch on a deliberately different architecture: no conference bridge and **no keypad step**. Instead, a single `dial` with a `target []` array, a per-target confirm hook, and a Redlock-serialized claim (§5). Acceptance became

automatic on answer — the fastest pickup wins the lock — because forcing every agent to “Press 1” added friction to the fast path the queue exists to optimize. The data model was kept intentionally lean: the rebuild sat on top of the **first-generation pull queue’s** existing config table and Redis-lock claim (§2), adding **zero new columns** to that table and introducing only the agent-login and login-history tables it genuinely needed.

The honest twist. Removing “Press 1” is the right call for agent UX, but it is exactly what makes answer supervision hard: without a human keypress, the system infers “a human answered” from the carrier’s answer signal — and some routes signal “answered” before a person is actually on the line. That is the primary hardening item before production (§14). The team knows the discarded design would have side-stepped it, and still chose the better UX, which frames the remaining work precisely rather than pretending it away.

9. Technology stack

Layer	Technology
Programmable voice / media	Jambonz (ring-all dial with target[] , per-target confirm hook, dial-end action hook)
Telco / SIP	Bandwidth (also used for auxiliary BXML hold audio)
Backend	TypeScript on the Hono web framework, running on the Bun runtime
Persistence	MySQL via the Drizzle ORM
Distributed coordination	Redis — Redlock for the claim race; a SET NX key for fallback dedup
Realtime	NATS pub/sub → WebSocket server → agent / supervisor UI
API surface to UI	trRPC procedures (status, login toggle, supervisor agent management)

On the database: MySQL, accessed through the **Drizzle ORM** in TypeScript from a shared schema package. The platform runs a **dual connection strategy** selected by environment — **PlanetScale’s serverless driver** against MySQL in production, and the `mysql2` driver against a local MySQL (MySQL 9) for development — both using the same Drizzle schema, with reads directable at a replica connection.

10. Data model

The feature is backed by the platform’s relational MySQL schema (Drizzle ORM). Its defining trait is restraint: the rebuild **reused the first-generation pull queue’s per-agent routing-config table without adding any columns**, and introduced only the tables it genuinely needed — **agent login state** (who is currently logged in and therefore eligible to be rung) and a **login-history archive** (the live row is moved here on logout). A per-number enablement flag lives on the phone-number configuration, and the platform’s central call-state table carries the routing fields this feature populates (status, claimed user, bridged leg id + answer time, a claimable flag, a queue-hold timestamp). Per-organization and per-number configuration is stored as **JSON, not columns**.

The full table-by-table schema — keys, indexes, and the JSON configuration shape — is maintained in ASRP’s engineering reference. Deeper technical detail is available under NDA.

11. Reliability & operations

- **Two-level feature gate, enforced everywhere.** Both an organization flag and a per-number flag must be true, and the gate is re-checked at three independent points — call entry, claim confirm, and fallback — so a mid-call configuration change cannot strand a call on an inconsistent path. Rejections are logged with a clear reason.
- **Deterministic claim under race.** A Redis Redlock keyed to the call serializes concurrent answers; the “already claimed” check inside the lock makes a second winner impossible. Losing legs are cleanly hung up.
- **Idempotent fallback.** The fallback path is guarded by a Redis `SET NX` key (2-hour TTL) so duplicate or retried dial-end webhooks route the caller at most once. With no fallback destination configured, the caller hears a graceful message rather than dead air, and the missing route is logged.
- **Graceful “no agents” behavior.** A call can be marked claimable even when no agent is logged in, allowing a manual claim from the UI instead of a hard failure.
- **Structured lifecycle logging.** Every stage emits a structured event (dial initiation, dial initiated, confirm accepted/rejected, claimed, fallback routing/skipped, unable-to-connect, caller-hangup cleanup) for traceability.

12. Design parameters

The routing behavior is driven by explicit, **configurable** parameters rather than hard-coded constants — the number of agents rung per call, the ring timeout, the claim-lock lease, and the fallback-deduplication window are all tunable per organization or per number, with sensible defaults. These are configuration defaults verified against the delivered source, **not** measured performance figures — no real-traffic metrics have been captured yet (see §13). The exact default values are recorded in ASRP’s engineering reference (available under NDA).

13. Results & outcome

The ring-all path, first-answer claim, feature gate, agent presence (login/logout/history), realtime events, and fallback are all implemented and exercised by an automated test suite. The suite covers the scenarios that matter: no logged-in agents → hold for a manual claim; one agent → claim via the confirm hook; two agents racing → exactly one wins the lock; dial failure → fallback; caller hangup while ringing → clean cleanup with no claim or fallback.

Smart Queue is positioned as a **pilot and a candidate to replace the incumbent external-PBX hunt-group**. As of the engagement it had **not** been measured on production call volume, so this page makes no claims about time-to-answer, drop rate, or utilization against the old queue — those numbers do not exist yet, and we would rather say so than invent them.

14. Maturity & roadmap

Honest split of what is shipped versus designed:

LIVE / built and tested (behind the feature gate): ring-all dial, deterministic first-answer claim under the Redlock, the two-level feature gate, database-backed agent presence (login/logout/history), realtime queue/claim/status events, idempotent fallback, and the manual-claim path when no agent is logged in. All covered by the automated suite.

Primary hardening focus before a production cutover: answer-detection edge cases on outbound carrier/PBX routes. Because acceptance is automatic on answer (no keypad — see §8), the “connected” transition must correspond to a *real human* answer on every route type; making that robust across carrier and PBX routes is the main item standing between pilot and production.

Designed / roadmap (next increments): - **Configuration UI** — enablement, group assignment, ring timeout, max agents, and fallback are database/JSON-driven today; a self-service admin UI is the natural next step. - **Durable queue-event history** — a dedicated event store (attempted targets, ring/no-answer/busy outcomes, claim races, answer-rate metrics) to unlock analytics and observability dashboards beyond today’s structured logs. - **Supervisor / device parity** — device selector, registration status, last-selected device, and per-device on-call presence, to reach full parity with the incumbent external queue. - **State-progression refinement** — richer lifecycle progression surfaced to the UI (incoming → waiting → connected → completed) across all hangup and fallback permutations.

15. What ASRP delivered

ASRP owned the **entire Smart Queue subsystem** on top of the platform’s existing call model — the ring-all verbs, eligibility queries, claim service, feature gate, agent-login schema and CRUD, realtime events, tRPC procedures, and UI toggles.

- Designed and implemented the in-house **ring-all routing** (single Jambonz `dial` with a `target[]` array + per-target confirm hook).
- Built the **first-to-answer claim** with a Redis Redlock and an **idempotent fallback** (Redis `SET NX`).
- Authored the **two-level feature gate** (organization + per-number flags), enforced at entry, claim, and fallback.
- Designed the **agent-presence data model** (login state + history archive) and its CRUD, plus the eligibility query joining groups, per-agent routing config, and login state.
- Implemented **real-time WebSocket/NATS events** and the **tRPC procedures** and **UI toggles** for agent self-login and supervisor management.
- Owned the full **POC → revert → rebuild** lifecycle, including the deliberate redesign away from the conference-bridge / keypad-confirm prototype toward the lock-based automatic-claim model (§8).
- Wrote the **automated test suite** covering the core routing scenarios.

16. FAQ (for client conversations)

Q: Is this an external PBX or a third-party ACD? No — it is a fully in-house queue built on Jambonz verbs plus the platform’s own database and Redis. It was specifically built to bring queue logic, agent presence, and claim behavior inside the platform rather than delegating them to an external PBX hunt-group.

Q: Does the agent press a key to accept the call? No. All logged-in agents ring simultaneously and acceptance is automatic on answer via a per-target confirm hook. The fastest answer wins. An earlier prototype used “Press 1 to accept”; it was reverted and the mechanism was rebuilt without a keypad step

(§8).

Q: What stops two agents from both getting the same call? A Redis distributed lock (Redlock) keyed to the call. The first answered leg claims it inside the lock; any later leg finds the call already claimed and is hung up. Exactly one winner, deterministically.

Q: What happens if no agent answers? An idempotent fallback path routes the caller to a configured fallback destination; if none is configured, the caller hears a graceful “unable to connect” message. Duplicate or retried webhooks cannot double-route.

Q: How do you turn it on or off? Two independent flags — one at the organization level and one per phone number. Both must be enabled, and the gate is re-checked at call entry, at claim, and at fallback.

Q: What database did you use? MySQL via the Drizzle ORM in TypeScript. Production uses PlanetScale’s serverless driver; local development uses the `mysql2` driver against MySQL. Redis backs the claim lock and fallback deduplication.

Q: Is it in production? It is a pilot — a candidate to replace the incumbent external-PBX hunt-group — with answer-detection hardening as the main item before a production cutover. No real-traffic metrics have been captured yet (§13).

Q: How is it tested? An automated suite covers the key scenarios: no logged-in agents (hold for manual claim), one agent (claim via confirm hook), two agents racing (exactly one wins the lock), dial failure (fallback), and caller hangup while ringing (clean cleanup).

Anonymized project description prepared by ASRP for portfolio use. Client name, proprietary source paths, internal identifiers, and unresolved-defect / risk-register content are intentionally omitted from the body. Deeper technical detail is available under NDA.
